Online Training
C# 6.NET Online Training Program
C# .NET.NET Core Training
Microservices Online Training Program
Microsoft Azure Training
Angular Training

Introduction & Environment
C# Basics
Free Compiler Mono
Introduction to Programming Language
Free Compiler Programs
Hello World
Different Types of Applications
Programming Methodologies
Algorithm, Pseudocode, Flowchart, Programs
Introduction to .NET Framework
.NET Framework Architecture and Components
Introduction to .NET Programming Language
How to Download and Install Visual Studio on Windows
Creating First Console Application using Visual Studio
.NET Developer Roadmap for 2021
Coding Standard Best Practices

C# .NET Basics
Data Types
Implicit Conversion of C# Program
Methods and Properties of Console Class in C#
Data Types in C#
Variables in C#
Constants in C#
Operators in C#
Control Flow Statements in C#
If-Else Statements in C#
Nested If-Else Statements
Switch in C#
While Loop in C#
Do While Loop in C#
For Loop in C#
Nested For Loop in C#
Continue Statement in C#
Goto Statement in C#
Functions in C#
User-Defined Functions in C#
Call By Value and Call By Reference in C#
Recursion in C#
Command Line Arguments in C#
Command Line Arguments in C#
Strong in C#
Enum Datatype in C#
Static vs Dynamic Methods in C#
Arrays and Read Only in C#
Jagged Arrays in C#
Multidimensional Arrays Initializing Method in C#
Overrides Equals Method in C#
Difference Between Convert.ToString and Tostring() Method in C#
Difference Between Tostring() and Cast Operator in C#
Private Constructors in C#
Destructors in C#
Garbage Collection in .NET Framework
Difference Between Finalize and Dispose in C#
Access Specifiers in C#
Encapsulation in C#
Abstraction in C#
Inheritance in C#
Types of Inheritance in C#
How to use Inheritance in Application Development
C# and Multi Inheritance in .NET
Generalization and Specialization in C#
Abstract Class and Abstract Methods in C#
Abstract Class and Abstract Properties in C#
Interfaces Definition in C#
How to Use Abstract Classes and Interfaces in Application
Interfaces in C#
Interface Member Functions and Answers in C#
Interface Runtime Examples in C#
Multiple Inheritance in C#
Multiple Inheritance Runtime Example in C#
Polymorphism in C#
Method Overloading in C#
Operator Overloading in C#
Method Overriding in C#
Method Hiding in C#
Partial Class and Partial Methods in C#
Sealed Class and Sealed Methods in C#
Extension Methods in C#
Static Class in C#
Variable Reference and Instance of a Class in C#

OOPs Real Time Example
Real time Example of Encapsulation Principle in C#
Real Time Example of Abstraction Principle in C#
Real Time Examples of Inheritance in C# Code
Real Time Examples of Polymorphism in C# Code
Real Time Examples of Interface in C# Code

Exception Handling
Exception Handling in C#
Exceptions Types in C#
Throw Exceptions in C#
Throw Exceptions in C#
Exceptions Hierarchy in C#
Exceptions Hierarchy in C#
Error Handling in C#
Custom Exceptions in C#
Custom Exceptions Example in C#

Events, Delegates and Lambda Expressions in C#
Delegates Structure of Events, Delegates and Lambda Expressions
Rules of Events, Delegates and Lambda Expressions
Delegates in C#
Multicast Delegates in C#
Delegates Real Time Example in C#
Generic Delegates in C#
Generic Delegates in C#
Anonymous Methods in C#
Lambda Expressions in C#
Events in C# with Examples

Multi Threading
Multi Threading in C#
Thread Class in C#
How to Pass Data to Thread Function in Type Safe Manner in C#
How to Determine Data from a Thread Function in C#
Join Method and Isalive Property of Thread Class in C#
Thread Synchronization in C#
Lock in C#
Monitor Class in C#
Mutex Class in C#
Semaphore Class in C#
SemaphoreSlim Class in C#
Thread Pool in C#
Advantages and Disadvantages of Multithreaded Application
Thread Pool in C#
Timer Callback Delegate in C#
Timer Callback in C#
Thread Pooling in C#
Thread Pooling in C#
Thread Cancellation in C#
How to Terminate a Thread in C#
Timer Thread Cancellation in C#
How to Sleep a Thread Threadsleep in C#

File Handling
File Handling in C#
Directory Class in C#
DirectoryInfo class in C#
StreamWriter in C#
File Class in C#
TextWriter and TextReader in C#
File Stream
BinaryWriter and BinaryReader in C#
StreamWriter and StreamReader in C#
File Class in C#
TextWriter and TextReader in C#
File Stream
BinaryWriter and BinaryReader in C#
StreamWriter and StreamReader in C#
Filehelper Class in C#
Directoryinfo Class in C#
Directory and Import Excel Data in C#

Asynchronous Programming
Introduction to Concurrency
Async and Await in C#
Task in C#
How to Return a Value from Task in C#
How to Execute Multiple Tasks in C#
How to Limit Number of Concurrent Tasks in C#
Task Cancellation in C# Using Cancellation Tokens
How to Create Synchronous Method using Task in C#
Race Patterns in C#
Only One Pattern in C#
How to Control the Result of Task in C#
Task Based Asynchronous Programming in C#
Cancel Task in C#
InterTask in C#
How to Cancel a Non-Cancellable Task in C#
Cancellation Tokens in C#
How to Cancel Asynchronous Stream in C#

Parallel Programming
Task Parallel Library in C#
Parallel for in C#
Parallel Foreach Loop in C#
Parallel Invoke in C#
Pipeline Design of Parallelism in C#
Parallelism in C#
How to Cancel Parallel Operations in C#
Async Methods, Thread Safety and Race Conditions in C#
Interlocked in Lock in C#
Parallel LINQ in C#
PLINQ#Ordering in Asynchronous Programming
in #ParallelProgramming in C#

AutoMapper
AutoMapper in C#
AutoMapper Complex Mapping in C#
How to Map Complex Type to Primitive Type using AutoMapper in C#
AutoMapper Reverse Mapping in C#
AutoMapper Conditional Mapping in C#
AutoMapper Ignore Method in C#
Fixed and Dynamic Values in Destination Property in AutoMapper

Optional Parameter, Indexers and Enums
Optional Parameter Optional Parameters in C#
Indexers in C#
Indexers Real Time Example in C#
Enums in C#

.NET Framework Architecture
.NET .NET Framework
Common Language Runtime (CLR) in .NET Framework
.NET Program Execution Process
Intermediate Language (CIL)IL#.S.A.O#C Code in .NET
Common Type System in .NET Framework
Common Language Specification in .NET
Managed and Unmanaged Code in .NET Framework
Assembly DLL EXE in .NET Framework
Namespaces in .NET Framework
Strong and Weak Assemblies in .NET Framework
How to Install an Assembly in the GAC in .NET Framework
DLL Hell Problem and Solution in .NET Framework

Ref, Out, Params and Reflection
Reflection in C#
Dynamic Type in C#
Var Keyword in C#
Var vs Dynamic in C#
Dynamic vs Reflection in C#
Dynamic Keyword in C#
Ref vs Out in C#
Named Parameters in C#

C# 7 new Features
C# 7 New Features
Enhancement in Out Variables in C# 7
Pattern Matching in C#
Digit Separators in C# 7
Local Functions in C#
Defining Tuples in C# 7
Local Functions in C# 7
Ref Returns and Ref Locals in C#
Expression Bodied Members in C#
Throw Expression in C#
Async Main in C#

C# 8 More Features
C# 8 New Features
Nullability Solution in C#
Default Interface Methods in C#
Pattern Matching in C#
Using Declarations in C#
Static Local Functions in C#
Disposable Ref Structs in C#
Nullable Reference Types in C#
Asynchronous Streams in C#
Asynchronous Disposable in C#
Indices and Ranges in C#
Null Coalescing Assignment Operator in C#
Unmanaged Constructed Types in C#
Stackalloc in C#

Most Popular C# Books
Pro C# 7 Microsoft Framework
Books
Pro C# 7 Microsoft Framework Books using C#
Pro C# 7 Microsoft Framework Books using C#

Common Language Runtime in .NET Framework

Common Language Runtime (CLR) in .NET Framework:

In this article, I am going to discuss the **Common Language Runtime (CLR) in .NET Framework**. Please read our previous article before proceeding to this article as we have discussed the **.NET Framework**. At the end of this article, you will understand all about CLR with Examples. Before we understand Understanding Common Language Runtime in .NET Framework, let us first understand how a .NET application is compiled and run.

How is a .NET Application Compiled and Run?

In order to understand how exactly a .NET Application is compiled and run, please have a look at the following image.

First, the developer has to write the code using any .NET one supported programming language such as C#, VB, or VC. Then the respective language compiler will compile the source code and generate something called Microsoft Intermediate Language (MSIL) or Intermediate Language (IL) code. For example, if the programming language is C# then the compiler is C#C and if the programming language is VB, then the compiler will be VBC. This Intermediate Language (IL) code is half-compiled code. It, partially compiled code cannot be executed directly by the Operating System. To execute this Microsoft Intermediate Language (MSIL) or Intermediate Language (IL) code on our machine, the .NET Framework provides something called Common language Runtime (CLR), which takes the responsibility to execute our Microsoft Intermediate Language (MSIL) or Intermediate language (IL) Code.

The CLR takes the IL or Intermediate language code and gives it to something called the JIT (Just-In-Time) Compiler which is part of the CLR. The JIT compiler reads each and every line of the IL code and converts it to machine-specific instructions (i.e. into binary machine code) which will be executed by the underlying Operating System. The CLR in .NET Framework provides the runtime environment to execute the .NET Application.

What is Intermediate Language (IL) Code in .NET Framework?

The Intermediate Language or IL code in .NET Framework is a half-compiled or partially compiled or CPU independent partially compiled code and this code can not be executed by Operating System.

Why Partial Compiled Code or Why Not Fully Compiled Code?

As a developer, you might be thinking about why the respective language compiler generates partially compiled code or why not fully compiled code i.e. machine code or binary code in .NET Framework when we compile the source code. The reason is that the code is platform independent and platform independent .NET Code is going to run on (for example, Windows XP, Windows 7, Windows 10, Windows 11, Windows Server etc.).

In other words, we don't know what operating system is going to run our application. We also don't know the CPU configuration, Machine Configuration, Security Configuration, etc. If the corresponding operating system where we want to run our .NET Applications, the Microsoft Intermediate language (MSIL) or Intermediate language (IL) code is compiled to machine compiled, and at runtime, the Microsoft Intermediate language (MSIL) or Intermediate language (IL) code is compiled to machine code, then it will be a CPU specific code. This code will not be based on underlying Operating System, CPU, Machine Configuration, etc. But the CLR in .NET Framework.

Common Language Runtime (CLR) in .NET Framework:

CLR is the heart of the .NET Framework and it contains the following components:

1. Security Manager
2. JIT Compiler
3. Memory Manager
   1. **Garbage Collector**
   2. **Exception Manager**
   3. **Common Type System (CTS)**
   4. **Common Type System (CTS)**

Let us discuss what each of these components does in detail in .NET Framework.

Security Manager:

There are basically two components to managing security. They are as follows:

1. **Code Access Security:**
2. **C# Code Verification:**

These two components are basically used to check the privileges of the current user and then check whether the user is allowed to access the assembly or not. The Security Manager also checks what kind of rights or authorities the code has and whether it is safe to be executed by the Operating System or not. So, basically these types of security checks are managed by the Security Manager Component of CLR in .NET Application.

JIT Compiler:

The JIT (Just-In-Time) Compiler is responsible for Converting the MSIL code into Native Code (Machine Code or Binary code) that is going to be executed by the Operating System. The Native Code (Machine Code or Binary code) is platform independentcode by the system hardware. JIT compiles the code just before the execution and then saves the translation in memory.

Memory Manager:

The Memory Manager component of CLR in the .NET Framework allocates the necessary memory for the variables and objects that are to be used by the application.

Garbage Collector:

When a lot of application runs, lots of objects are created. At a given point in time, it is possible that some of these objects are not used by the application. So, **Garbage Collector** in **.NET Framework** is nothing but a **Reset Routine** or you can say it is a **Background Thread** that runs periodically and tries to identify what objects are not being used currently by the application and de-allocates the memory of those objects.

Exception Manager:

The **Exception Manager** component of CLR in the .NET Framework redirects the control to execute the catch or finally blocks whenever an exception has occurred at runtime. If we have not handled the Runtime Exception, then the Exception Manager will throw the exception and eventually terminate the application execution at the time where the exception has occurred.

Common Type System (CTS) in .NET Framework:

The .NET Framework supports many programming languages such as C#, VB.NET, J#, etc. Every programming language has its own data type. But the programming language data types cannot be understood by other programming languages. But, then can be situations where we need to communicate between two different programming languages. For example, we need to write a code in the VB.NET language and that code may be called from C# language. In order to ensure smooth communication between these languages, the most important thing is that they should have a **Common Type System (CTS)** which ensures that language-specific datatypes defined in two different languages get compiled into a Common Language Specification.

CLR in .NET Framework will execute all programming languages's data types. This is possible because CLR has its own language specification and common set of data types. Common Type System (CTS) is a common set of data types which are supported by all .NET Supported Programming Languages. Every language compiler follows the language specification of CLR and generates the MSIL code. This language specification of CLR is common for all programming languages and this is known as **Common Language Specifications (CLS).**

In the next article, I am going to discuss the **.NET Program Execution** process. Here, in this article, I try to explain the Common Language Runtime (CLR) in C#.NET Framework. I hope you enjoy this CLR in C# article.

DOT NET TUTORIALS
Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3000+ articles in the 22 years career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new things.

Previous Lesson
.NET .NET Framework
Next Lesson
.NET Program Execution Process

5 thoughts on "Common Language Runtime in .NET Framework"

ABHIT
MAY 12, 2020 AT 12:04 AM
I think u they compiler is cos.exe not csc.exe
Well great article !!!
Reply
DOT NET TUTORIALS
MAY 12, 2020 AT 8:02 AM
Yes, the compiler is cos.exe. This is a typo error. We have corrected this. Thank you for identifying the typo error.
Reply
SAMI
SEPTEMBER 26, 2020 AT 1:49 AM
great artcilleThank you man.....
Reply
PARTHA
FEBRUARY 3, 2021 AT 2:20 PM
Is CIL code is related to the CPU dependent while in this CPU independent. Could you please clarify this?
Reply
SUJAY
FEBRUARY 11, 2021 AT 3:33 PM
IL Code is related to the CPU dependent while in this CPU independent. Could you please clarify this?
Reply
Leave a Reply
Your email address will not be published. Required fields are marked *
Comment *
Name *
Email *
Website
Post Comment